

Trading Simulator

Database design document

Group 19	Prateek Thota 2024AAPS0254H	Aditya Sanadhaya 2024AAPS0263H	Shreyas Khunteta 2024AAPS0302H	Muhilan Selvakumar 2024AAPS0309H	11 October 2026
9 TABLES	4 VIEWS	6 TRIGGERS	4 PROCEDURES	6 INDEXES	1316 SEED ROWS

1 Problem and requirements

Trading Simulator is a small stock exchange. Traders keep cash in trading accounts and buy and sell shares of listed companies. Buy and sell orders are matched like on a real exchange: best price first, and at the same price, the earlier order first. Matching, payment and moving shares are all done by stored procedures in PostgreSQL, so the database enforces the rules about money.

Users and what they can do

There is one kind of user, the trader. A trader can

- ▶ search the listed instruments by ticker or company name, filter them by sector, and see each one's order book (open buy and sell quantity at each price) and recent trades;
- ▶ place a limit order to buy or sell. It is matched straight away against the other side, and any unfilled part stays on the book;
- ▶ cancel an order that is still on the book, which releases the cash held for it;
- ▶ deposit and withdraw cash;
- ▶ see their portfolio (holdings, average cost, market value, unrealised profit or loss), their trade history and their cash statement;
- ▶ keep named watchlists of instruments;
- ▶ run reports across all accounts (portfolio valuations, most active traders, and more).

Assumptions

- ▶ One exchange and one currency (rupees). A trade settles the moment it happens.
- ▶ No borrowing. Every buy is paid for in cash and every sell is backed by shares already held, so there is no margin trading and no short selling.
- ▶ Each company has exactly one listed instrument.
- ▶ There is no sign-up or login. Users and accounts come from the seed script, and the application acts as whichever account is selected.
- ▶ The schema allows MARKET orders, but this version only matches LIMIT orders.

Business rules enforced by the database

- 1 A buy order is accepted only if the account's free cash (cash_balance minus reserved_cash) covers quantity × limit price. That amount is reserved immediately.
- 2 A sell order is accepted only if the account holds enough shares that are not already offered in its other open sell orders.
- 3 Orders match by price priority, then by time priority.
- 4 One incoming order may fill against several resting orders, the last of which may be filled only in part.
- 5 An account never trades with itself.
- 6 Each fill updates both orders, records the trade, moves cash and shares and writes one ledger row for each side, all in a single transaction.
- 7 Trades and ledger rows are permanent. Any UPDATE or DELETE on them is rejected.
- 8 The average cost of a holding is recalculated on every buy as a weighted average.
- 9 Only free cash can be withdrawn. Cash reserved for open buy orders can't be.

2 EER model

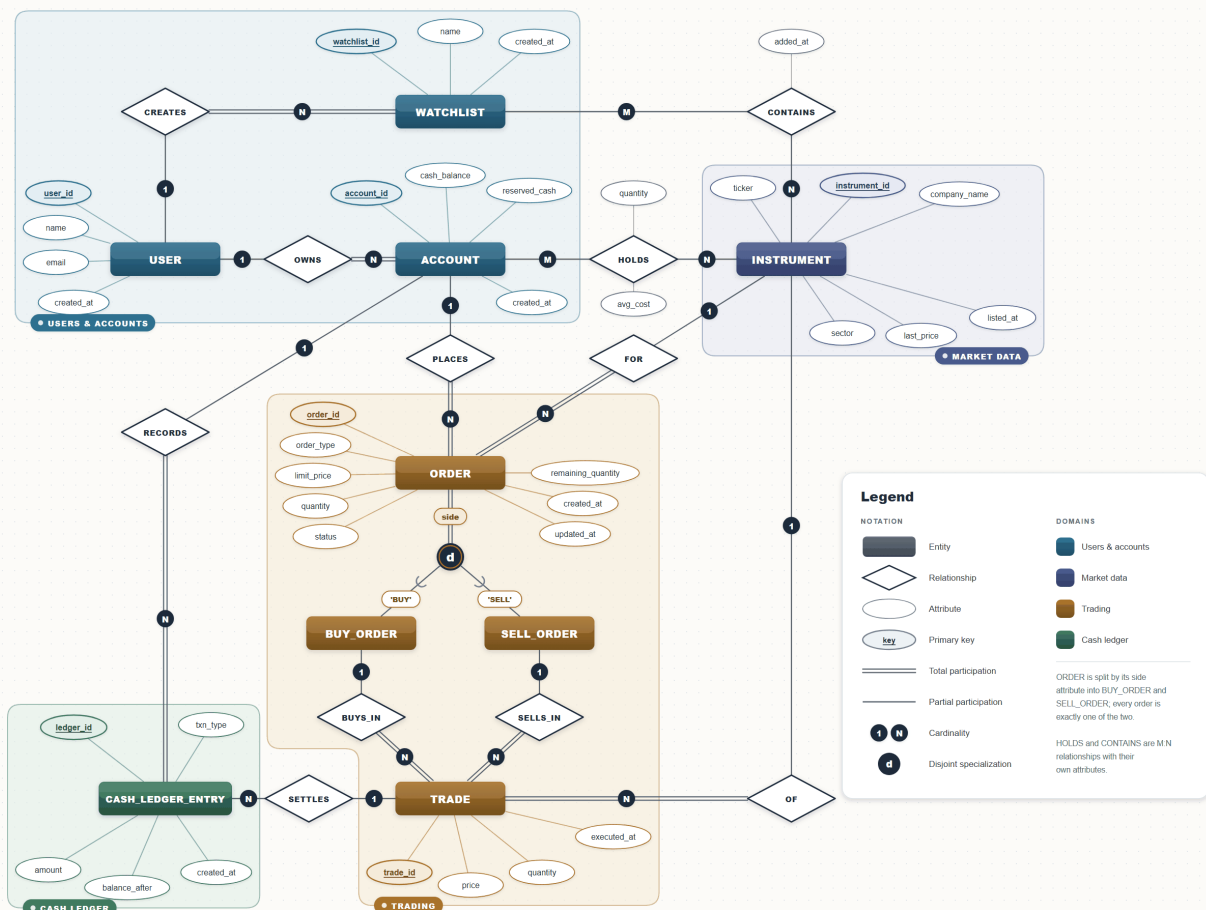


Figure 1. EER diagram in Chen notation. Double lines mark total participation and underlined attributes are keys.

The model has seven entity types (USER, ACCOUNT, WATCHLIST, INSTRUMENT, ORDER, TRADE, CASH_LEDGER_ENTRY), two subtypes of ORDER, and eleven relationship types.

Specialization. ORDER is specialized on its side attribute into BUY_ORDER and SELL_ORDER. It is disjoint, because an order is either a buy or a sell, and total, because side is mandatory. We added it because a trade is exactly one buy order (BUYS_IN) matched with one sell order (SELLS_IN). The subtypes have no attributes of their own, so both are stored in the orders table with side as the discriminator.

Relationships with attributes. HOLDS (account to instrument) carries quantity and avg_cost, and CONTAINS (watchlist to instrument) carries added_at. Both are M:N, so each becomes a table keyed by the two foreign keys. There are no weak entities, since every entity type has its own key.

Cardinality and participation. USER-ACCOUNT is 1:N because one person can have several accounts. Every account belongs to a user, but a user can exist without an account, so participation is total only on the account side. An order may never be filled, so ORDER takes part in BUYS_IN and SELLS_IN partially. A ledger entry points to a trade only when it is a trade payment; deposits and withdrawals have none.

3 Relational schema

Each entity type became a table with a surrogate key. Each 1:N relationship became a foreign key on the N side, declared NOT NULL wherever participation is total. HOLDS and CONTAINS became holdings and watchlist_items. The specialization is one table with side as discriminator, and BUYS_IN and SELLS_IN became the two foreign keys buy_order_id and sell_order_id on trades. Price history isn't a separate table: every price change is a trade, so it is the view v_price_history on trades. That gives nine tables.

Primary keys are underlined; foreign keys are in italics.

```

users (user_id, name, email, created_at)
accounts (account_id, user_id, cash_balance, reserved_cash, created_at)
instruments (instrument_id, ticker, company_name, sector, last_price, listed_at)
orders (order_id, account_id, instrument_id, side, order_type, limit_price, quantity, remaining_quantity, status,
created_at, updated_at)
trades (trade_id, buy_order_id, sell_order_id, instrument_id, price, quantity, executed_at)
holdings (account_id, instrument_id, quantity, avg_cost)
cash_ledger (ledger_id, account_id, txn_type, amount, balance_after, related_trade_id, created_at)
watchlists (watchlist_id, user_id, name, created_at)
watchlist_items (watchlist_id, instrument_id, added_at)

```

Candidate keys. Besides the primary keys: users.email, instruments.ticker, watchlists(*user_id*, name), and cash_ledger(*related_trade_id*, txn_type) among trade rows.

Domains. Money is NUMERIC(14,2) and prices are NUMERIC(12,2). These are exact decimals, because floating point can't store amounts like 0.10 exactly. Quantities are INT, and codes such as side and status are short VARCHARs limited by CHECK. Timestamps are TIMESTAMPTZ, so they are correct in any time zone.

Foreign keys

TABLE	FOREIGN KEY	REFERENCES AND NOTES
accounts	user_id	users. NOT NULL
orders	account_id, instrument_id	accounts, instruments. Both NOT NULL
trades	(buy_order_id, instrument_id) (sell_order_id, instrument_id)	orders(order_id, instrument_id). Composite, so a trade's instrument must be the instrument of both its orders
holdings	account_id, instrument_id	accounts, instruments
cash_ledger	account_id, related_trade_id	accounts; trades (nullable, empty for deposits and withdrawals)
watchlists	user_id	users
watchlist_items	watchlist_id, instrument_id	watchlists (ON DELETE CASCADE), instruments

Integrity constraints

Every column is NOT NULL except `instruments.sector`, `orders.limit_price` (empty for market orders) and `cash_ledger.related_trade_id`.

TABLE	CONSTRAINTS BEYOND KEYS AND NOT NULL
users	email unique
accounts	cash_balance ≥ 0 , reserved_cash ≥ 0 , reserved_cash $\leq$ cash_balance
instruments	ticker unique, last_price > 0
orders	side in (BUY, SELL); order_type in (LIMIT, MARKET); limit_price > 0 and present exactly when the order is LIMIT; quantity > 0 ; $0 \leq$ remaining_quantity $\leq$ quantity; status in (OPEN, PARTIALLY_FILLED, FILLED, CANCELLED) and consistent with remaining_quantity; (order_id, instrument_id) unique, as the target of the composite keys
trades	price > 0 , quantity > 0 , buy_order_id $\neq$ sell_order_id; append-only (trigger)
holdings	quantity ≥ 0 , avg_cost ≥ 0
cash_ledger	txn_type in (DEPOSIT, WITHDRAWAL, TRADE_BUY, TRADE_SELL); sign of amount matches the type; trade rows must name their trade and other rows must not; balance_after ≥ 0 ; (related_trade_id, txn_type) unique; append-only (trigger)
watchlists	(user_id, name) unique; name not blank

4 Normalization

1NF. Every column holds a single atomic value. There are no arrays, lists or JSON columns.

2NF. Only holdings and watchlist_items have composite keys. In holdings, quantity and avg_cost describe one account's position in one instrument, so they need the whole key. In watchlist_items, added_at is when that instrument was added to that watchlist. Every other table has a single-column key, so 2NF holds.

3NF and BCNF. In every table, every determinant is a candidate key, with one exception. In trades, instrument_id is determined by buy_order_id, which is not a key of trades (one order can have

several trades), so this is a transitive dependency. We kept it because the price-history view, the last-price trigger and the trade-history queries all use it, and it saves a join each time. It can't become inconsistent, because the composite foreign keys on (buy_order_id, instrument_id) and (sell_order_id, instrument_id) reject a trade whose instrument doesn't match its orders. On the EER diagram this column is the relationship OF. Apart from this, the schema is in BCNF.

Stored derived values. Some columns are calculated from other rows but stored anyway, because they are read all the time. This doesn't break any normal form, but it is redundancy. They are cash_balance and reserved_cash in accounts, remaining_quantity and status in orders, last_price in instruments, quantity and avg_cost in holdings, and balance_after in cash_ledger (a running balance, like on a bank statement). The app never writes them directly. They only change inside the procedures (place_order, cancel_order, deposit_cash, withdraw_cash) or the trade trigger.

5 Design decisions

The matching engine is in the database. compute_fill_plan is a read-only function. A CTE gets the resting orders that an incoming order can match, and a window function (SUM() OVER (ORDER BY price, time ...)) keeps a running total of their quantity, which tells us how much to take from each. place_order then applies this plan in one transaction. Doing this in PostgreSQL means all money movement happens in one place, and a fill either happens completely or not at all.

Concurrency. place_order and cancel_order first take an advisory lock on the instrument, so orders for one instrument are matched one at a time and two buyers can't take the same sell order. Then they lock every account involved with SELECT ... FOR UPDATE, always in increasing account_id order, so they can't deadlock.

Reservations. A buy reserves cash in reserved_cash. A sell checks shares against what is already offered in other open sells. Paying for a fill and releasing its reservation are done in one UPDATE, because PostgreSQL checks reserved_cash ≤ cash_balance after every statement. With two separate statements, a buyer with little spare cash would fail halfway through a valid trade. If a buy fills below its limit price, only the lower price is charged and the rest becomes free cash again.

Deposits and withdrawals. deposit_cash and withdraw_cash are procedures so the balance and its ledger row always change together. Withdrawing reserved cash is refused by the procedure, and the reserved_cash ≤ cash_balance check would refuse it anyway.

Triggers and views. A trigger on trades updates instruments.last_price, and two more make trades and cash_ledger append-only. At first we had a price_history table filled by a trigger, but every row was just a copy of a trade, so we replaced it with a view.

Not included. Stop orders, immediate-or-cancel orders, circuit breakers and login, to keep the project a manageable size.